ignoClient — complete reference

The Python client for ignoAPI. Three layers, and they behave differently enough that picking the wrong one is the usual source of surprise:

Layer	Talks to	Connection	Use it for
Variables (`igo_*`)	one server, `SERVER_HOST`	kept open per thread	ordinary reads and writes
Cache (`fast_*`)	the same server, in the background	one sync thread	control loops, 10 ms and faster
Modules (`create*`)	any machine, named per call	opened and closed each call	provisioning, setup scripts

```
import ignoClient as c
```

> *The file is* `ignoClient.py` *— capital* **C**. *Windows will import* `ignoclient` *too; Linux will not.*

Contents

- Connecting
- Variables
- The cache
- Finding machines
- Modules
- createAutoClient
- createComClient
- createModbusMaster
- createModbusSlave
- createSyncIgnoAPI
- createFTP
- combineVar
- listModules / deleteModule / createModule
- The wire protocol
- Errors
- Recipes

Connecting

The variable and cache layers talk to **one** server, set by module constants:

```
c.SERVER_HOST = "ignoapi_abi.local"   # default "127.0.0.1"
c.SERVER_PORT = 8899                  # default from hardwareCSV.API_PORT
```

Set these **before** the first call — the socket is opened lazily and then reused, so changing the host afterwards does not move an already-open connection.

Constant	Default	Meaning
`SERVER_HOST`	`"127.0.0.1"`	the machine the `igo_*` calls use
`SERVER_PORT`	`8899`	its TCP port
`CONNECT_TIMEOUT`	`2.0`	seconds to establish a connection
`RECV_TIMEOUT`	`3.0`	seconds to wait for a reply
`CACHE_SYNC_INTERVAL`	`0.01`	seconds between background cache syncs

Address machines by mDNS name, not by IP. `ignoapi_abi.local` follows the machine; `192.168.1.27` follows the DHCP lease. A script holding an IP quietly configures the wrong machine after a reboot, or none at all. Use an IP only where a machine has no `ADDRESS` set.

The module functions ignore `SERVER_HOST` entirely — every one of them takes the machine as its first two arguments.

Variables

One socket per OS thread, reconnected automatically. A failed call retries once, then raises.

`igo_create(name, value)`

Create it, or overwrite it if it is already there.

```
c.igo_create("target_speed", 0.0)
c.igo_create("robot_name", "AGV-3")
c.igo_create("waypoints", [1, 2, 3])
```

`igo_write(name, value)`

Write to a variable that **must already exist**. Prints an error and leaves the store untouched if it does not — deliberately, so a typo cannot silently invent a variable nothing reads.

```
c.igo_write("target_speed", 1.4)
c.igo_write("no_such_var", 1)   # [ignoClient] write 'no_such_var': does not exist. Use create first.
```

`igo_create_or_write(name, value)`

Both, in one call. The one you usually want.

```
c.igo_create_or_write("target_speed", 1.4)
```

`igo_read(name, fallback=None)`

Returns the value, or `fallback` if the variable does not exist.

```
speed = c.igo_read("target_speed", 0.0)
```

Raises `RuntimeError` if the **server** cannot be reached — a missing variable and a missing server are different failures and are reported differently.

`igo_read_all()`

The whole store, one round trip, as a `dict`. `{}` on failure.

```
for name, value in sorted(c.igo_read_all().items()):
    print(f"{name:30} {value!r}")
```

`igo_bulk_write(data)`

Many variables, one round trip. Use this instead of a loop of `igo_write` — a loop pays the round trip per key.

```
c.igo_bulk_write({
    "a_x": 12.5,
    "a_y": -3.2,
    "a_id": None,
})
```

`igo_delete(name)`

```
c.igo_delete("scratch")
```

Deleting a **module** variable this way removes only the row. To take the child process and its companions with it, use `deleteModule`.

The cache

For loops that read the same values every few milliseconds. A background thread mirrors the whole store locally, so reads cost nothing.

`start_cache_sync()`

Call once, at startup. Starts the sync thread.

`fast_read(key, fallback=None)`

Reads the local mirror. No network, no latency.

`fast_write(key, value)`

Updates the mirror immediately and queues a push to the server, so the value is readable by `fast_read` before it has reached the server.

`get_cache_age()`

Seconds since the last successful sync. **Check it.** If the server dies the cache keeps answering with values that get older and older, and nothing else tells you.

```
c.SERVER_HOST = "ignoapi_abi.local"
c.start_cache_sync()

while True:
    if c.get_cache_age() > 1.0:
        stop_everything()                # the mirror is stale; do not steer on it
        continue
    x = c.fast_read("a_x", 0.0)
    c.fast_write("cmd", compute(x))
    time.sleep(0.01)
```

> *The docstring on* `fast_read` *says the cache refreshes every 50 ms. The constant is* `CACHE_SYNC_INTERVAL = 0.01`*, so it is 10 ms. The comment is stale, not the code.*

Finding machines

`findIgnoAPIs(timeout=1.5)`

Broadcasts `ignoAPI?` on UDP 8899 and returns every answer:

```
for m in c.findIgnoAPIs():
    print(m["ip"], m["name"], m["port"])
# 192.168.1.10 ignoapi_abi.local 8899
# 192.168.1.14 ignoapi_komal.local 8899
```

Each entry is `{"ip": str, "name": str, "port": int}`. `name` **is empty** when that machine has no `ADDRESS` set — fall back to `ip` in that case.

Sent from every network interface, so a machine on a second adapter (a VPN) is found too, and each machine answers with the address that reaches it from where the question came.

Modules

Everything the right-click menu can create, a script can create. All of them share the same first three arguments:

```
create<Thing>(host, api_port, name, ...)
              └──┬──┘  └──┬──┘  └─┬─┘
     the machine to      its TCP   the VARIABLE to create
     configure           port      on that machine
```

Two notes that will save an argument-order bug:

- `name` **is required and has no default.** Every module *is* a variable — an autoclient with no name has nothing to write into.
- **The target port is** `api_port`**, not** `port`**.** Every module has a `port` of its own — a serial port, an FTP port, the port an autoclient polls — and the two would collide.

Every function returns the server's message string, and raises `RuntimeError` if the machine refuses or cannot be reached. Anything not named keeps the value the create form would have shown, and `**extra` passes any other field through by name.

`runatstartup` accepts `True`, `"TRUE"`, `"true"`, `"1"`, `"yes"`, `"on"` — anything else is false.

createAutoClient

A TCP poller: `name` polls another server and stores what it answers.

```
createAutoClient(host, api_port, name, server_ip, server_port, command,
                 runatstartup=False, **extra)
```

Argument	Meaning
`server_ip`	the machine to poll (mDNS name preferred)
`server_port`	its port
`command`	what to send each poll, e.g. `"0x01"`

```
c.createAutoClient("ignoapi_abi.local", 8899, "mag",
                   "ignoapi_komal.local", 8899, "0x01",
                   runatstartup=True)

# a slower poll, and its own log
c.createAutoClient("ignoapi_abi.local", 8899, "mag",
                   "ignoapi_komal.local", 8899, "0x01",
                   poll_interval_sec="1.0", log_file="mag.log")
```

Extra fields: `mac`, `poll_interval_sec`, `connect_timeout_sec`, `read_timeout_sec`, `log_file`.

createComClient

A serial reader. Creates `<name>_tx` beside it — type into that and the reader transmits it.

```
createComClient(host, api_port, name, com_port, baud=115200,
                command=None, runatstartup=False, **extra)
```

```
c.createComClient("ignoapi_abi.local", 8899, "scale", "COM3", 9600)
```

Extra fields: `probe`, `poll_interval_sec`, `connect_timeout_sec`, `read_timeout_sec`, `log_file`.

Leaving `command` out keeps the default listen command, which is what you want for a device that talks on its own.

createModbusMaster

MODBUS RTU master. Creates four command rows beside it — `readReg`, `writeReg`, `readCoil`, `writeCoil` — which is how you talk to the bus.

```
createModbusMaster(host, api_port, name, com_port, baud=38400, parity="N",
                   stopbits=1, bytesize=8, runatstartup=False, **extra)
```

```
c.createModbusMaster("ignoapi_abi.local", 8899, "plc", "COM4", 19200)

# then drive it by writing the command rows
c.igo_create_or_write("plc_readReg", "1,40001,2")
```

Choices: `baud` 4800–115200, `parity` `N`/`E`/`O`, `stopbits` 1/2, `bytesize` 8/7. Extra fields: `timeout_sec`, `poll_interval_sec`, `log_file`.

`createModbusSlave`

MODBUS RTU slave. Each register in the map becomes its own variable, and writing one changes what a master reads back.

```
createModbusSlave(host, api_port, name, com_port, register_map,
                  slave_id=1, baud=38400, parity="N", stopbits=1,
                  bytesize=8, runatstartup=False, **extra)
```

```
c.createModbusSlave("ignoapi_abi.local", 8899, "panel", "COM5",
                    r"D:\maps\panel.csv", slave_id=7)
```

`register_map` **is a path on the target machine**, not on yours — it is read there. Extra fields: `persist_csv`, `timeout_sec`, `log_file`.

`createSyncIgnoAPI`

Links `name` to the same variable on other machines.

```
createSyncIgnoAPI(host, api_port, name, peers, remote_var="",
                  topology="star", mode="0x01", runatstartup=False, **extra)
```

Argument	Values	Meaning
`peers`	list, dict or string	the other machines
`remote_var`	`""`	empty means *the same name over there*
`topology`	`"star"` (default), `"delta"`	see below
`mode`	`"0x01"` LEVEL, `"0x02"` PULSE	how long a received value stays

```
# star — every peer talks to abi and to nobody else
c.createSyncIgnoAPI("ignoapi_abi.local", 8899, "abiserver",
                    ["ignoapi_komal.local", "ignoapi_aditya.local"])

# delta — every machine can address every other, abi relaying
c.createSyncIgnoAPI("ignoapi_abi.local", 8899, "abiserver",
                    ["ignoapi_komal.local", "ignoapi_aditya.local"],
                    topology="delta")
```

An unrecognised `topology` or `mode` falls back to `star` / `0x01` rather than half-working.

What the choice creates:

Row	star	delta
`abiserver_tx`	to everybody	to everybody
`abiserver_rx`	most recent from anyone	most recent from anyone
`abiserver_<machine>_tx`	one per peer, when 2+ peers	one per peer, always
`abiserver_<machine>_rx`	one per peer, when 2+ peers	one per peer, always
peers hear each other	no	yes, relayed by the hub

A **single-peer star** gets neither per-machine box — there, "everybody" and "that machine" are the same place.

In a delta the peers still hold one connection each, to the hub, so the hub *is* the relay: peer-to-peer traffic stops when the hub does. That is what lets a delta need no setup at all on the other machines, and it is the price.

Extra fields: `mac`, `pulse_ms`, `poll_interval_sec`, `connect_timeout_sec`, `read_timeout_sec`, `log_file`.

`createFTP`

An FTP server sharing one folder.

```
createFTP(host, api_port, name, folder, ftp_port=2121, userid="",
          password="", sharing=None, runatstartup=False,
          read_only=False, **extra)
```

```
# anonymous, unshared — the short call
c.createFTP("ignoapi_abi.local", 8899, "drop", r"C:\Folder")

# with a login, read-only, and the link pushed to two machines
c.createFTP("ignoapi_abi.local", 8899, "drop", r"C:\Folder",
            userid="line", password="s3cret", read_only=True,
            sharing={"ignoapi_komal.local": 8899,
                     "ignoapi_aditya.local": 8888})
```

`sharing` is who gets sent the `ftp://…` link. It takes any of:

```
{"ignoapi_komal.local": 8899, "ignoapi_aditya.local": 8888}
["ignoapi_komal.local", "ignoapi_aditya.local:8888"]
"ignoapi_komal.local:8899,ignoapi_aditya.local:8888"
```

Empty `userid`/`password` means **anonymous**, and anonymous plus writable means anyone who can reach the port can read *and overwrite* those files. Set `read_only=True` if the other machines only need to pull.

`folder` is a path on the target machine and must already exist there.

`combineVar`

Makes one variable a live join of others.

```
combineVar(host, api_port, name, *variables)
```

```
c.combineVar("ignoapi_abi.local", 8899, "row", "a", "b", "c")
# writes:  row = combine {a},{b},{c}
# ignoAPI then keeps row_combined filled in from a, b and c as they change
```

Returns the command string it wrote. Raises `ValueError` if given no variables.

A variable that does not exist leaves an **empty slot** rather than being dropped — the result is read by position, and a missing third value silently shifting the fourth into its place is the one failure that would not look like a failure.

`listModules / deleteModule / createModule`

```
c.listModules("ignoapi_abi.local", 8899)
# {'autoclient': ['mag'], 'com': [], 'ftp': ['drop'],
#  'modbusmaster': ['plc'], 'modbusslave': [], 'sync': ['abiserver']}

c.deleteModule("ignoapi_abi.local", 8899, "drop")
```

`deleteModule` works out the kind from the name — pass `kind=` only to be explicit. It removes the child process, the companion rows, the duplicated program, and (for a sync) the rows that sync created on the linked machines.

`createModule` is the one the others call, for a field the wrappers do not name:

```
c.createModule("ignoapi_abi.local", 8899, "autoclient", "mag",
               {"server_ip": "ignoapi_komal.local", "port": 8899,
                "command": "0x01", "poll_interval_sec": "0.5"})
```

Settings go in a **dict**, not keyword arguments, because a field called `port`, `host`, `kind` or `name` would otherwise collide with the target.

The wire protocol

Under all of it: TCP, one JSON object per line, one reply line each. No authentication. Anything can speak it:

```
import json, socket

def ignoapi(req, host="ignoapi_abi.local", port=8899, timeout=3):
    with socket.create_connection((host, port), timeout=timeout) as s:
        f = s.makefile("rwb")
        f.write((json.dumps(req) + "\n").encode())
        f.flush()
        return json.loads(f.readline().decode())
```

Action	Fields	Notes
`create`	`name`, `value`	makes it, or overwrites
`read`	`name`	errors if missing
`write`	`name`, `value`	**errors if missing**
`create_or_write`	`name`, `value`	
`append`	`name`, `value`	string concat, list append, number add
`delete`	`name`	
`read_all`	—	the whole store
`bulk_write`	`data`	many at once
`create_module`	`kind`, `name`, `fields`	
`delete_module`	`name`, optional `kind`	
`list_modules`	—	

Every reply is `{"status": "success", ...}` or `{"status": "error", "message": "..."}`.

There is also a discovery ping: any **non-JSON** line containing `ignoapi` gets a bare `OK` back. It only fires when the line is not valid JSON, so a real request whose value mentions "ignoAPI" is handled normally.

Reachability. ignoAPI binds `127.0.0.1` unless `API_NETWORK_HOST` is on, which binds `0.0.0.0`. Since there is no authentication, and `EXEC` / `EXECFLAG` are ordinary variables that make ignoAPI launch programs, treat anything that can reach the port as able to run code on that machine. Keep it on the LAN or behind a VPN — never port-forwarded.

Errors

Situation	What happens
Server unreachable (`igo_*`)	`RuntimeError` after one retry
Variable missing (`igo_read`)	returns `fallback`
Variable missing (`igo_write`)	prints an error, store unchanged
Module call refused	`RuntimeError` carrying the server's message
Module machine unreachable	`RuntimeError`
`combineVar` with no variables	`ValueError`

```
try:
    c.createFTP("ignoapi_komal.local", 8899, "drop", r"C:\NotThere")
except RuntimeError as e:
    print("could not set it up:", e)
```

Recipes

Set up a whole fleet from one script

```
import ignoClient as c

for m in c.findIgnoAPIs():
    host = m["name"] or m["ip"]          # the name where there is one
    if host == "ignoapi_abi.local":
        continue
    c.createSyncIgnoAPI(host, m["port"], "heartbeat",
                        ["ignoapi_abi.local"], runatstartup=True)
    print("linked", host)
```

Mirror one variable to every machine, then read them all back

```
c.SERVER_HOST = "ignoapi_abi.local"
c.igo_create_or_write("abiserver_tx", "shift change")

for m in c.findIgnoAPIs():
    print(m["name"], c.listModules(m["name"] or m["ip"], m["port"]))
```

Tear down everything a script created

```
host, port = "ignoapi_abi.local", 8899
for kind, names in c.listModules(host, port).items():
    for name in names:
        print("removing", kind, name)
        c.deleteModule(host, port, name)
```

A control loop that refuses to steer on stale data

```
c.SERVER_HOST = "ignoapi_abi.local"
c.start_cache_sync()

while True:
    if c.get_cache_age() > 0.5:
        c.fast_write("cmd", "STOP")
        continue
    c.fast_write("cmd", plan(c.fast_read("a_x", 0.0),
                             c.fast_read("a_y", 0.0)))
    time.sleep(0.01)
```